

DNS

Static and Dynamic DNS

Harsh Sarkar

M.Sc. Semester II, Computer Science, Visva-bharati

DNS: Domain Name System

Introduction

- # Any resource available in a network must be located in a computer connected to the network, and has to be accessed by using the network address of the computer.
- # DNS allows us to use high-level, readable names to identify computers in a network.
- # Advantages:
 - We do not need to remember IP addresses to access a computer.
 - An organization can change its IP address without needing to inform its users as the host name remains unchanged.

History

- # Back in the ARPANET days, a file, *hosts.txt*, listed all the computer names and their IP addresses.
- # Every night, all of the hosts would fetch it from the site at which it was maintained.
- # Disadvantages:
 - The size of the file would become too large.
 - Host name conflicts would occur constantly unless names were centrally managed, something unthinkable in a huge international network due to the load and latency.
- # The **Domain Name System** was invented in 1983 to address these problems.

Overview

- # To map a name onto an IP address, an application program calls a library procedure called the **stub resolver**, passing it the name as a parameter.
- # The stub resolver sends a query containing the name to a local DNS resolver called the **local resolver**.
- # The local resolver performs a **recursive lookup** for the name against a set of DNS resolvers.
- # Once the local resolver receives a reply containing the IP address, it caches the reply and forwards it to the stub resolver, which in turn returns it to the application.
- # DNS utilizes UDP to transfer queries and replies through port 53.
- # The dig tool in linux can be used to inspect the entire resolution process.

Overview

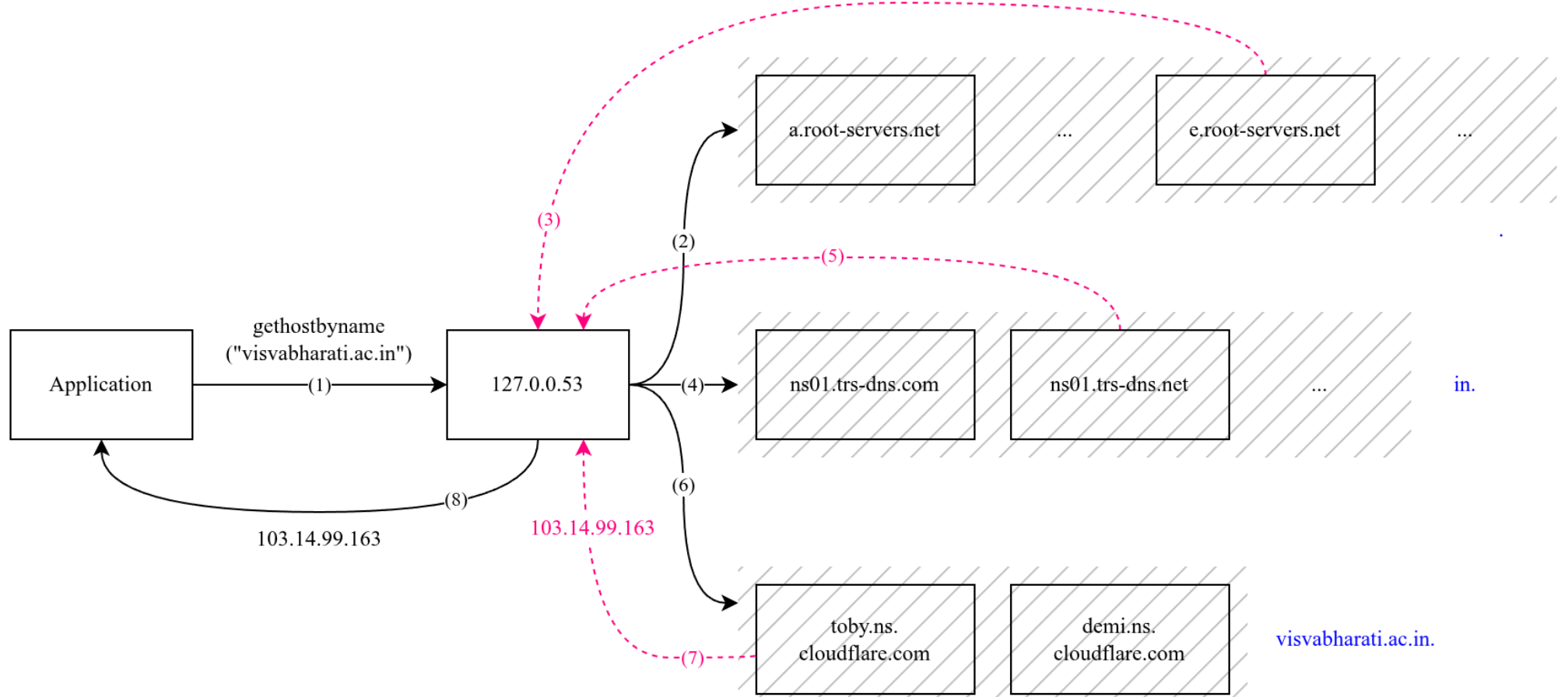


Figure 1: DNS resolution

DNS Namespace

- # Managing a large and constantly changing set of names is a nontrivial problem. In the postal system, name management is done by requiring letters to specify the country, state or province, city, street address, and name of the addressee.
- # For the Internet, the top of the naming hierarchy is managed by an organization called **ICANN** (Internet Corporation for Assigned Names and Numbers, *estd 1998*).
- # The Internet is divided into over 250 top-level domains (TLDs). Each TLD is further divided into sub-domains which are further partitioned and so on. In the hostname, the partitions are separated by the dot symbol.
- # There are two types of TDLs:
 - **Generic domains:** Domains introduced via applications to ICAAN.
 - **Country domains:** A domain for each country.

DNS Namespace

- # TLDs are run by **registrars** appointed by ICANN.
- # Getting a second-level domain requires going to the required registrar. The registrar checks if the desired name is available, and then the requester pays the registrar an annual fee and gets the name.
- # Each domain controls how it allocates the domains under it.
- # For example: To create a new hostname for our department, say *dcss.visvabharati.ac.in*, we only need to contact our university IT department and not the registrar of *in*.

DNS Namespace

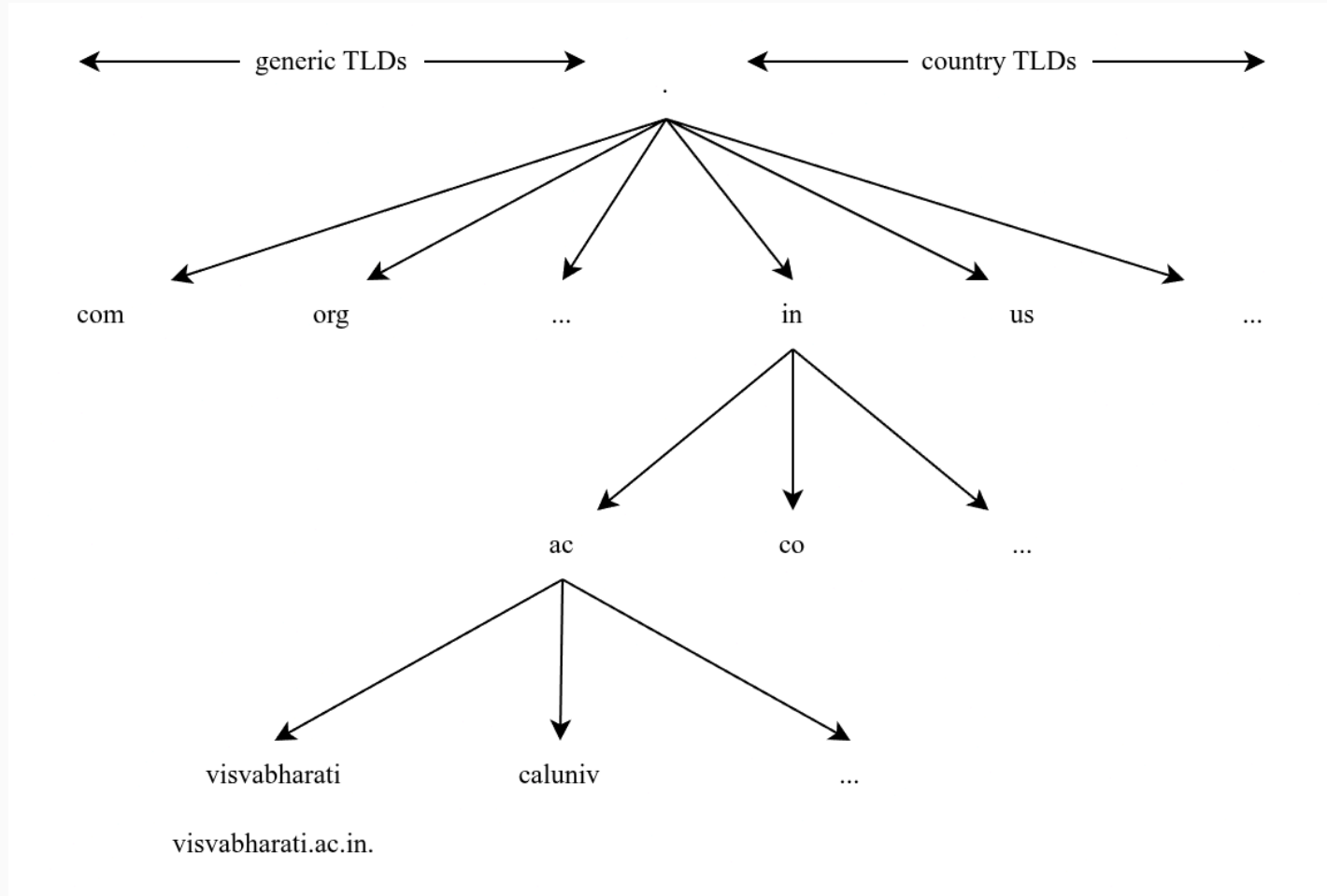


Figure 2: DNS namespace

Domain Resource Records

- # Every domain has a set of **resource records** associated with it.
- # The primary function of DNS is to map domain names onto resource records.
- # A resource record is a five-tuple consisting of:
 1. **Domain name:** Specifies the domain to which the record belongs.
 2. **Time to live:** Specifies how long the DNS cache will persist.
 3. **Class:** Specifies the network context. For most of the cases, it is *IN* and stands for *Internet*.
 4. **Type:** Tells what kind of record it is. Some important record types will be discussed later.
 5. **Value:** Contains some additional data about the record. As for the type of data, it depends on the record type.

Record Types

Type	Meaning	Value
<i>SOA</i>	Start Of Authority	Provides the name of the authoritative name server, email address of its administrator, a unique serial number, and various flags and timeouts.
<i>A</i>	Address	Holds a 32-bit IPv4 address of a host.
<i>AAAA</i>	IPv6 Address	Holds a 128-bit IPv6 address of a host.
<i>NS</i>	Name Server	Specifies a name server for the domain. A name server contains a copy of the DNS database of the domain.
<i>CNAME</i>	Canonical Name	Allow aliases to be created.
<i>PTR</i>	Pointer	Points to another name. It is used to associate a name with an IP address during reverse lookup .

Table 1: Commonly used record types

Record Types

Domain	TTL	Class	Type	Value
.	2754	IN	NS	<i>e.root-servers.net.</i>
<i>visvabharati.ac.in.</i>	900	IN	NS	<i>toby.ns.cloudflare.com.</i>
<i>visvabharati.ac.in.</i>	300	IN	A	<i>103.14.99.163</i>
<i>visvabharati.ac.in.</i>	1800	IN	SOA	<i>demi.ns.cloudflare.com.</i> <i>dns.cloudflare.com. 2404087003</i> <i>10000 2400 604800 1800</i>
<i>163.99.14.103.in-addr.arpa.</i>	3600	IN	PTR	<i>cloud.visvabharati.ac.in.</i>
<i>www.visvabharati.ac.in.</i>	300	IN	CNAME	<i>visvabharati.ac.in.</i>

Table 2: Examples of DNS records

Name Servers

- # To avoid the problems associated with having only a single source of information, the DNS name space is divided into nonoverlapping **zones**.
- # A zone usually contains one *primary name server*, which gets its information from a file on its disk, and one or more *secondary name servers*, which get their information from the primary name server.
- # Different types of name servers:
 - **Root name servers**: These servers maintain the root of the DNS hierarchy. Provides reference to TLDs. For more information on root name servers, go to www.iana.org.
 - **TLD name servers**: These servers maintain the top-level domains of the DNS hierarchy. For more information on TLD name servers, go to www.iana.org.
 - **Authoritative name servers**: They are the primary name servers of their respective zones and contain the DNS database.
 - **Local name servers**: Same as the *local resolvers* discussed earlier.

Name Servers

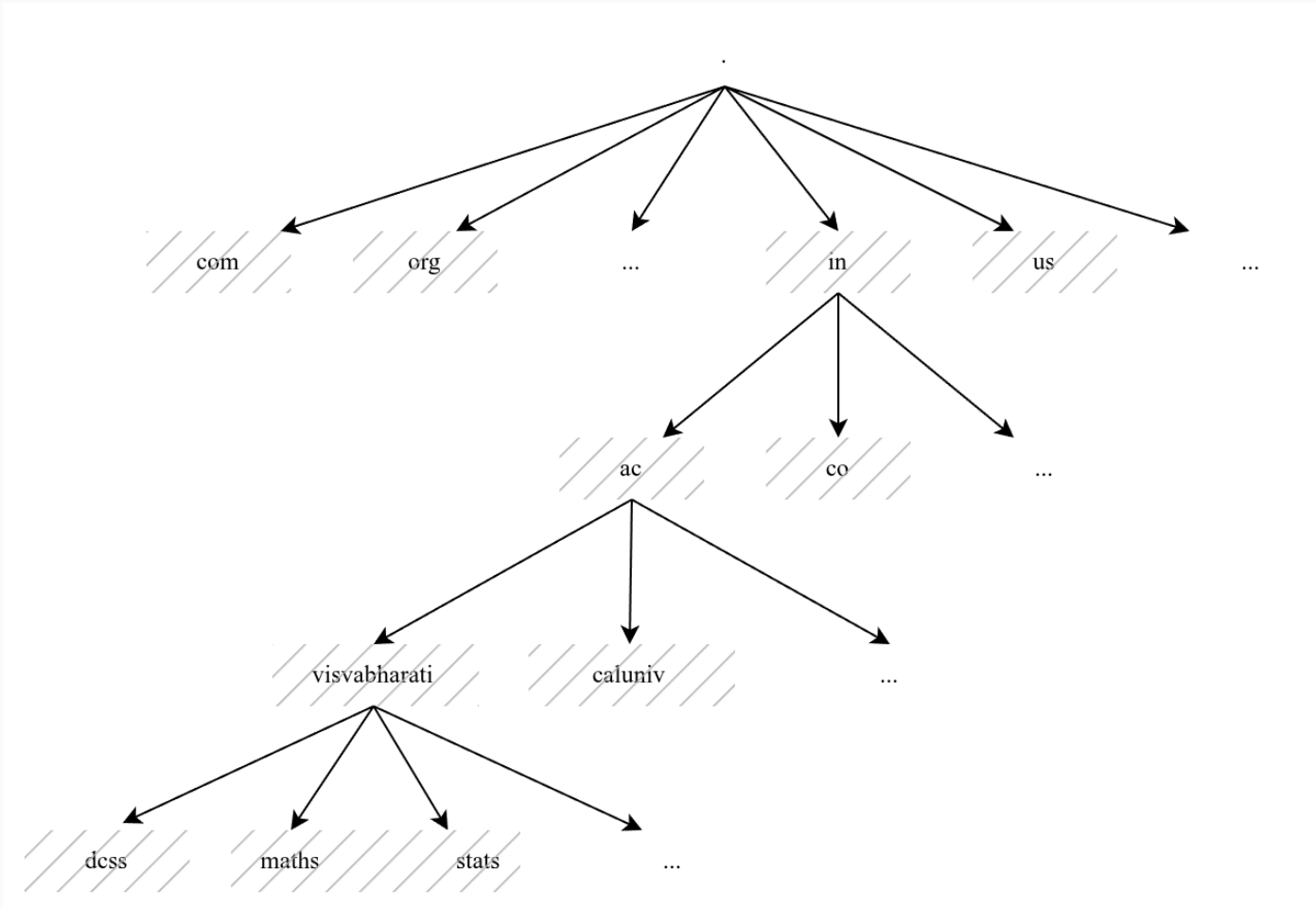


Figure 3: DNS namespace divided into zones

Name Servers

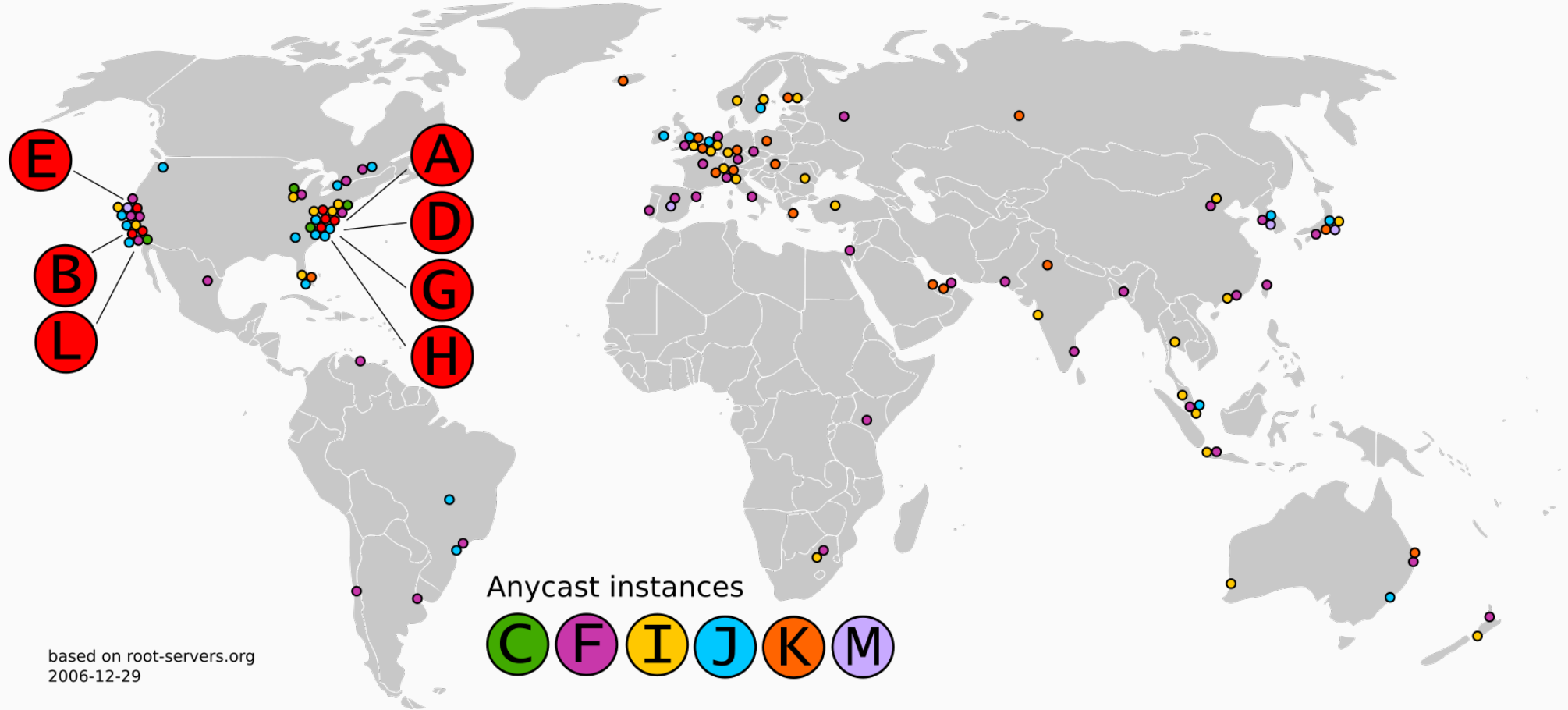


Figure 4: Root name servers, *source: wikipedia.org*

Name Resolution

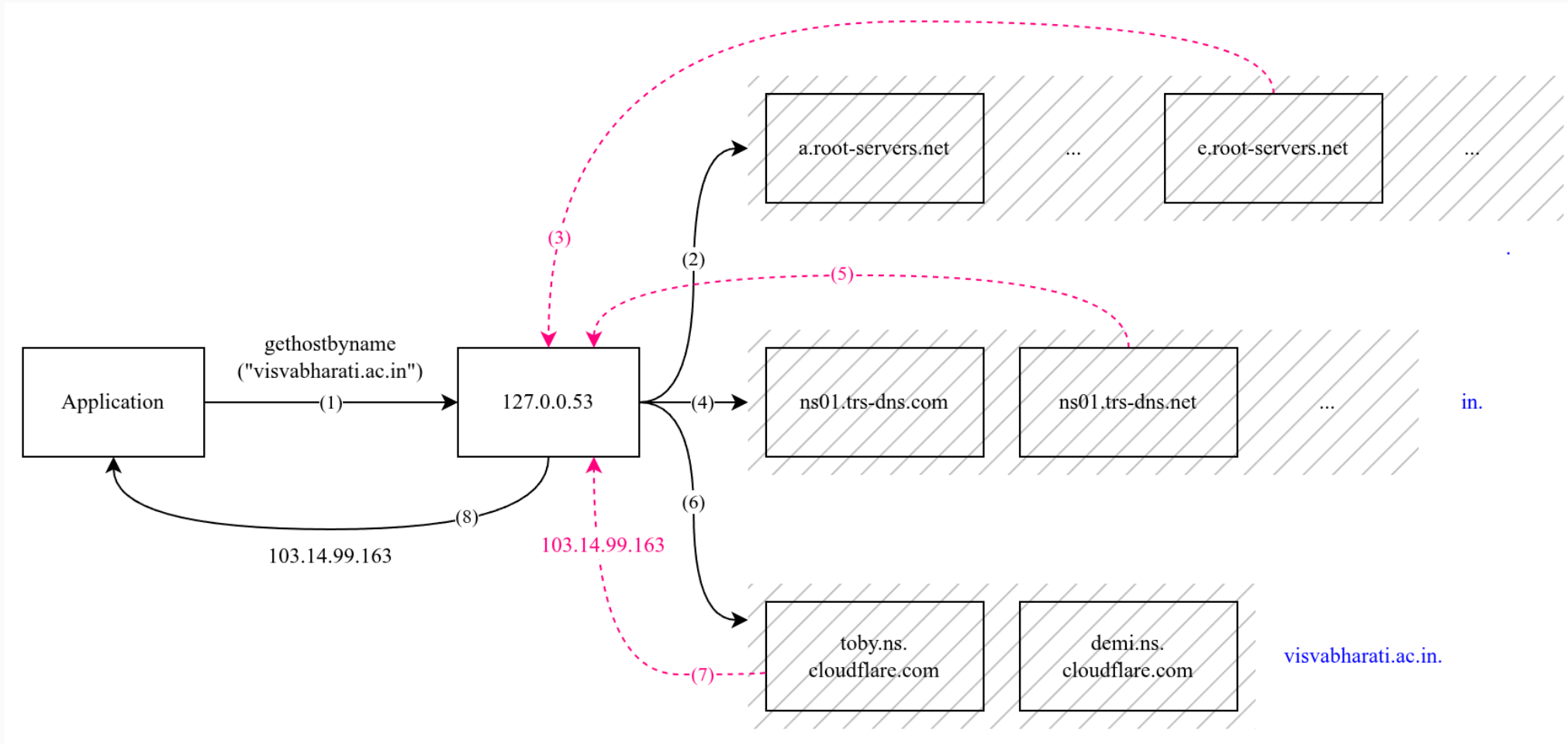


Figure 5: Recursive + iterative name resolution

Name Resolution

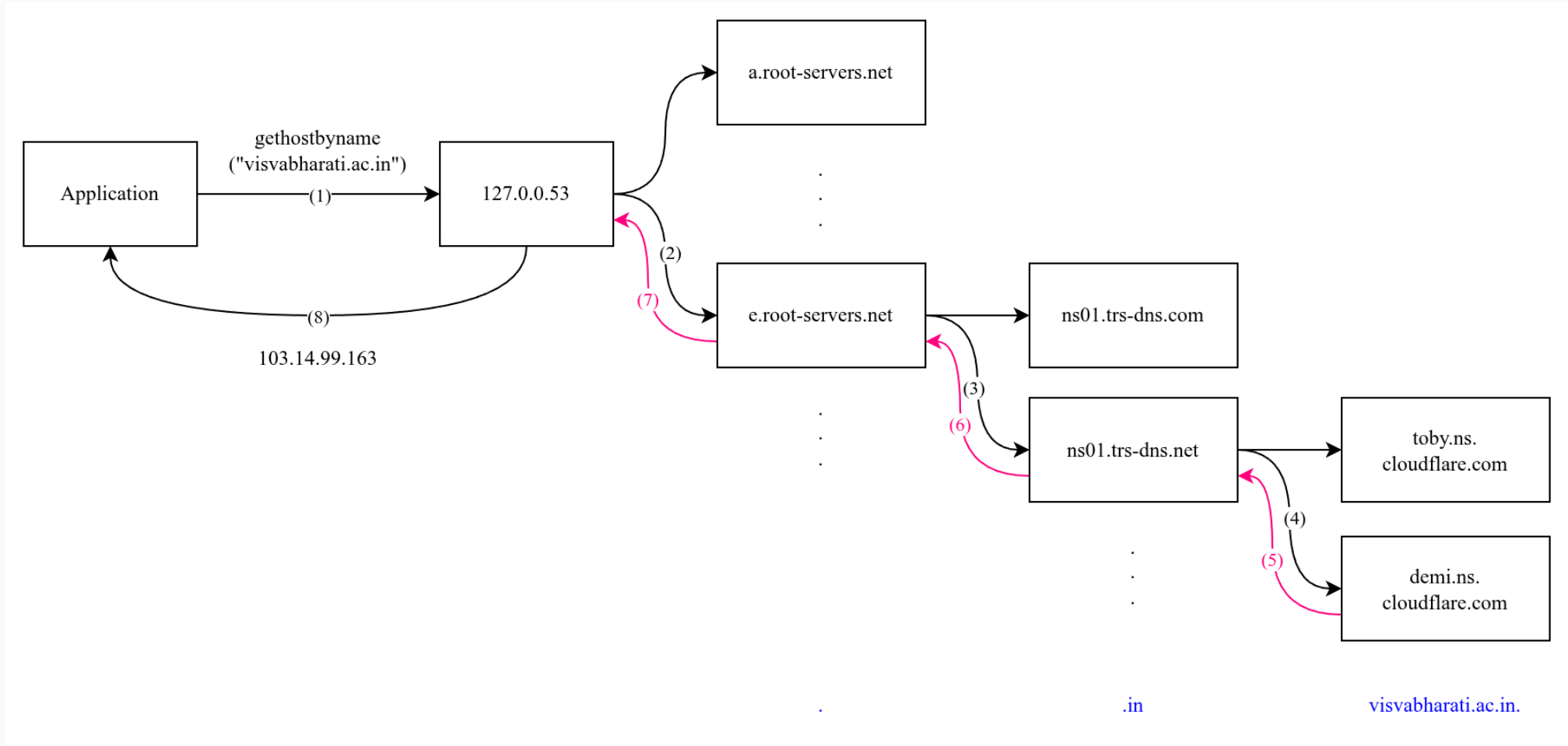


Figure 6: Recursive name resolution

Static and Dynamic DNS

Static IP Address

- # A static IP address is a manually configured identifier assigned to a device that remains consistent and cannot change across multiple network sessions.
- # Individuals do not typically need a static IP address, but businesses need them to host their own servers. Server and account administrators may also use whitelisted static IP addresses to manage sensitive assets that block most IP addresses.
- # **Pros**
 - Better name resolution
 - Anywhere and anytime access
- # **Cons**
 - Easy-to-track addresses
 - Difficult to change
 - Costly

Dynamic IP Address

- # ISPs temporarily assign dynamic IP addresses via the **Dynamic Host Configuration Protocol** (DHCP) server. This means an IP address can change every time a user reboots their router or system, and when the user connects to their ISP service.
- # When not in use, a dynamic IP addresses can be automatically assigned to another device. This makes dynamic IP addresses more suitable for home networks than large organizations.
- # **Pros**
 - Cost reduction
 - Enhanced security
 - Automatic configuration
- # **Cons**
 - Hosting problems
 - Problematic remote access

Static DNS

- # DNS maps hostnames to IP addresses.
- # A new hostname (e.g. *dcss.visvabharati.ac.in*) can be created under an existing hostname (e.g. *visvabharati.ac.in*) without informing name servers higher in the DNS hierarchy by adding a new record in the authoritative name server of the university.
- # But what if we want to change the IP address associated with the hostname? Furthermore, what if the IP address is dynamic and changes without our knowledge?

Dynamic DNS (DDNS)

- # In a **Dynamic DNS**, the record automatically changes when there is a change in the IP address of a host.
- # How is it done?
 1. **Provider API**
 - A DNS provider may provide an API to update our record. It also authenticates the API request to ensure no unauthorized person can update our record.
 - In most cases, the provider also provides a GUI application to simplify the process.
 2. **RFC 2136**
 - The RFC 2136 standard, titled *Dynamic Updates in the Domain Name System (DNS UPDATE)*, defines a protocol extension that allows DNS records to be added, modified, or deleted programmatically without manually editing zone files.
 - It especially focuses on validation and authentication so that there is no issue when updating DNS records.

DNS Providers



- # Cloudflare is more than just a DNS provider. Cloudflare is a platform that provides web services essential to build, deploy, secure and scale applications fast and with ease.
- # They provide a wide range of services including:
 - Database and storage
 - DDoS protection and rate limiting
 - CDN, DNS and load balancing
 - And a lot more...
- # Cloudflare was launched in 2010, and by 2019, Cloudflare's global network spanned 193 cities across more than 90 countries and it is still growing.
- # Cloudflare now powers 20% of the Internet!
- # You may read this blog at <https://blog.cloudflare.com> to learn more.

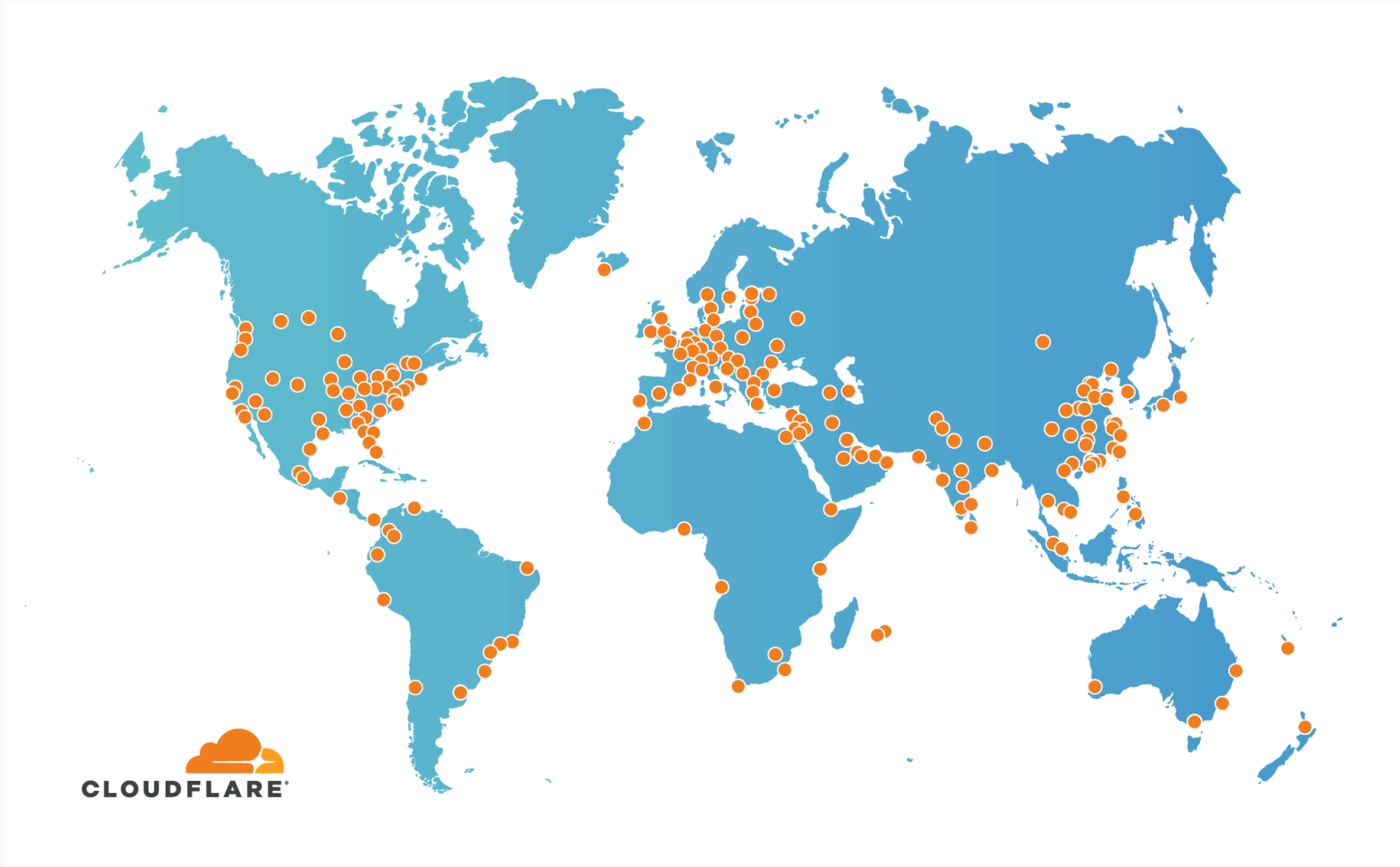


Figure 7: Distribution of Cloudflare data centers, *source: cloudflare.com*

- # No-IP provides DNS services for both personal and business use cases.
- # It offers the following services:
 - Dynamic DNS
 - Managed DNS
 - Fast and reliable DNS due to a global Anycast network of more than 150 servers worldwide.
- # No-IP provides a free DDNS domain along with an application to automatically update IP address. So, you may use your free domain by creating a No-IP account.



Figure 8: Distribution of No-IP DNS servers, *source: noip.com*

Duck DNS

what does the service actually do?

Duck DNS is a free service which will point a DNS (sub domains of duckdns.org) to an IP of your choice

who are we?

the team consists of two software engineers who each have worked in the industry for over 15 years

why do I need a dynamic DNS service

DDNS is a handy way for you to refer to a server/router with an easily rememberable name, where the servers ip address is likely to change when your router reconnects, or ec2 server reboots, its ip address is set by the provider of that connection, this means it may update at any time

why make a free DDNS service?

because we can, because before we started we couldn't, learning is fun

Proxy & Reverse Proxy

Proxy

- # Every time you open a website, your device talks directly to another server on the Internet. Your IP address, location and basic network details are visible to that server. In many cases, this is fine. But there are situations where you may want more control over how your requests travel across the Internet.
- # A **proxy** acts as an intermediary between you and the Internet.
- # From the server's point of view, it's the proxy that is making the request, not you.

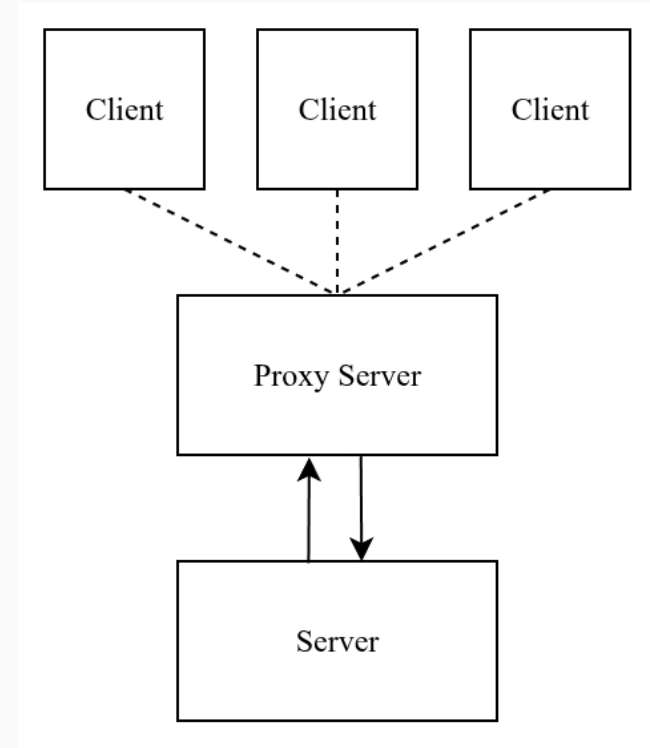


Figure 9: Proxy server as an intermediary

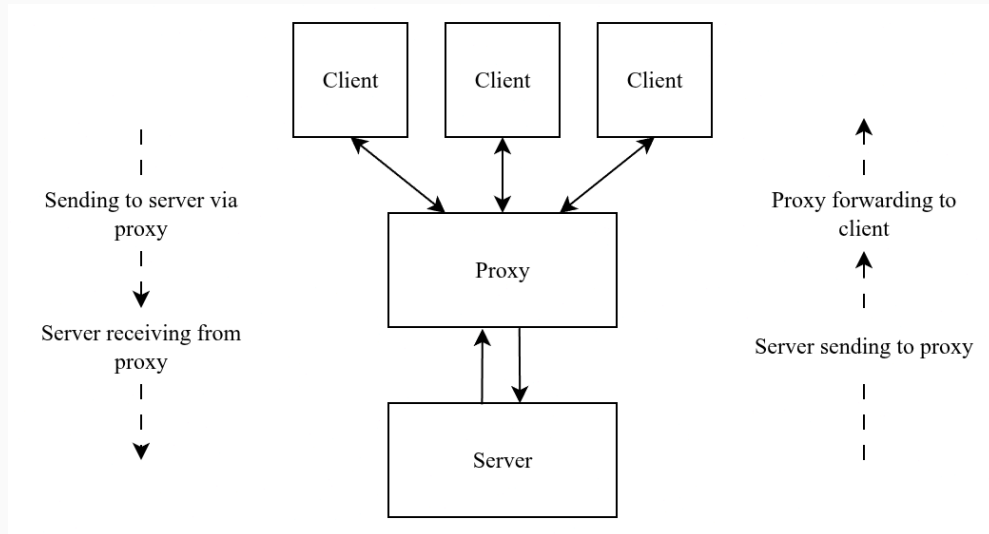
Internet Requests without a Proxy

- # When you type a website address into your browser, your computer resolves the domain name to an IP address using DNS. It then opens a connection directly to that server.
- # Your IP address is included as part of the network connection so the server knows where to send the response.
- # The server can log your IP address, infer your location, detect your network provider and apply rules based on that information.
- # Some websites restrict access by country. Others rate-limit or block traffic from specific IP ranges. In automated systems, repeated requests from the same IP are often flagged as suspicious.

Types of Proxies

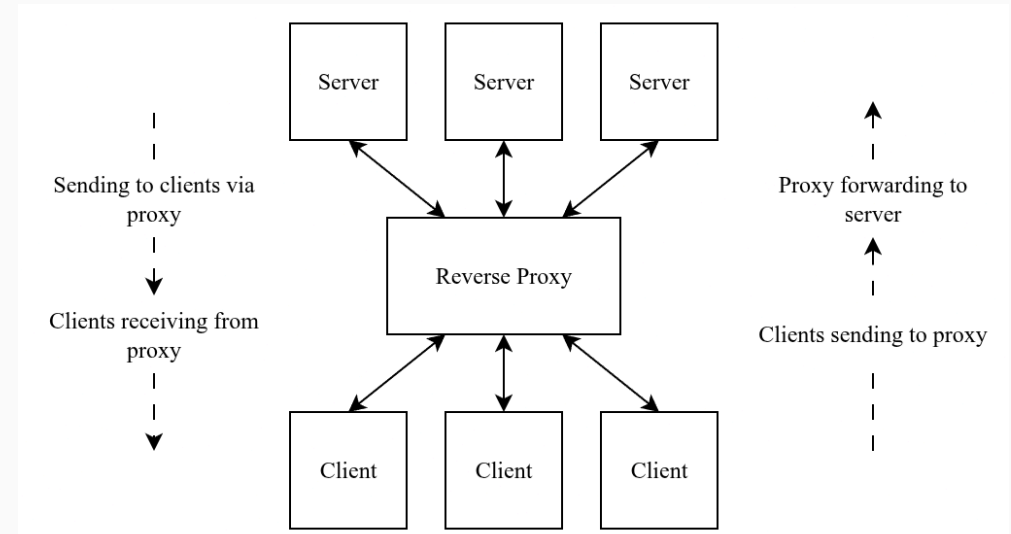
Forward Proxies

- # These are used by clients to access external resources.
- # Corporate networks often use forward proxies to control employee internet access.



Reverse Proxies

- # They sit in front of servers rather than clients.
- # Websites use reverse proxies to load balance traffic, terminate TLS and protect backend systems.



Proxy Use Cases

- # **To protect online identity:** One of the most common reasons to use a proxy is IP masking. By routing traffic through a proxy, your real IP address is hidden from the destination server.
- # **Bypass geographic restrictions:** Proxies are also used for geographic routing. If a service behaves differently in different countries, a proxy located in a specific region lets you see what users there experience.
- # **Automation and web scraping:** In automation and scraping systems, proxies are essential. Sending thousands of requests from a single IP is a fast way to get blocked. Rotating proxies distribute traffic across many IPs, reducing detection.
- # **Monitor and restrict traffic:** Companies use proxies to monitor, filter and log traffic. This helps with compliance, security and performance optimisation.

Reverse Proxy Use Cases

- # **Load balancing:** Popular websites have multiple servers distributed all over the world. By using a reverse proxy, the client sends requests only to the proxy, and the proxy forwards it to a suitable server among the pool of servers.
- # **Protection from attacks:** Since only the reverse proxy is visible to the public, an attacker can only attack the proxy server which has tighter security implemented.
- # **Caching:** A reverse proxy server can also cache content, resulting in faster performance.

- # **NGINX** is a free and open-source web server known for its high performance and low resource consumption.
- # Beyond serving static web files, it is heavily used as a reverse proxy, load balancer, API gateway and HTTP cache.
- # Key features:
 - **Reverse proxy:** Intercepts client requests and forwards them to backend application servers, hiding the backend structure for improved security and flexibility.
 - **Load balancer:** Distributes incoming network traffic across multiple servers, ensuring high availability and preventing any single server from becoming a bottleneck.
 - **Caching:** Efficiently serves static assets like images, HTML and CSS while offloading requests to application servers.
- # NGINX provides easy-to-understand documentation, so if you want to get started, visit: https://nginx.org/en/docs/beginners_guide.html

References



References

1. *Computer Networks, A Tanenbaum - 5th Edition*
2. *Computer Networking: A Top-Down Approach, Kurose - 6th Edition*
3. <https://chatgpt.com>
4. Linux man pages
5. https://en.wikipedia.org/wiki/Root_name_server
6. <https://www.fortinet.com/resources/cyberglossary/static-vs-dynamic-ip>
7. <https://www.cloudflare.com>
8. <https://www.noip.com>
9. <https://www.freecodecamp.org/news/a-developers-guide-to-proxy-servers>
10. <https://www.cloudflare.com/learning/cdn/glossary/reverse-proxy>
11. <https://nginx.org>